

The lux Display Singleton Lifecycle: a Z Specification

Formal model of the two-lock bind-race discipline

July 2026

1 Introduction

Two or more agents may, at the same instant, decide that no display is running and try to become the one display that owns a single Unix-socket path. Exactly one must win. The mechanism that arbitrates the race is a pair of advisory file locks — a *spawn lock* held end-to-end by **ensure** and **reap**, and a *bind lock* held by **setup** across the whole **probe** → **cleanup** → **bind** → **listen** critical section.

The subtlety the design must survive is the *bind window*. A socket that has been **bind**-ed but not yet **listen**-ed refuses connections; a liveness probe therefore reads it as *dead*. Without serialisation, a second agent probing during that window concludes the path is stale, unlinks the first agent's fresh socket, and binds its own — and now two displays believe they own the path. That is the round-2 defect fixed on branch `fix/lux-h29e-bind-race`.

This document models the lifecycle as a finite state machine over a small fixed set of agents, so that ProB can enumerate every interleaving and either confirm the five invariants or produce the exact offending trace.

2 Basic Types

[*AGENT*]

AGENT is the set of concurrent processes contending for the path. Two agents suffice to exhibit the two-winner race; three exercise lock contention more thoroughly. The carrier is bounded by ProB's `DEFAULT_SETSIZE`, so no explicit cardinality constant is needed.

3 Free Types

SOCKSTATUS ::= *sabsent* | *sbound* | *slistening* | *sstale*

The status of the *file at the socket path*, as a liveness probe would observe it. **sabsent**: no file. **sbound**: a socket is bound but **listen** has not been called — the bind window; a probe reads it dead. **slistening**: a live owner is accepting connections. **sstale**: a leftover socket file whose owner has died — a probe reads it dead.

PHASE ::=
idle | *hasSpawn* | *hasBind* | *probed* | *bound* |
listening | *serving* | *lostRace* | *exited* | *reaping*

Each agent's own progress. **idle** before it acts; **hasSpawn**/**hasBind** once it holds the corresponding lock; **probed** after it has observed the path; **bound** after a successful **bind**; **listening** after **listen**; **serving** once it has released the bind lock and is the running display; **lostRace** for an agent that observed a live owner or lost the bind; **exited** for a terminated agent; **reaping** for a reaper mid-teardown.

4 State

The state is flat: one schema, five components. Its predicate carries only *structural* invariants — cardinality bounds on the locks and owner, and the correspondence between the path's status and whether it has an owner. These hold in every design, correct or buggy, so they act as coherence constraints, not as

the safety properties under test. The safety properties (Section 9) are deliberately *absent* here, so that a violating state remains reachable and ProB can find it.

<i>Sys</i>	$ \begin{aligned} &socketStatus : SOCKSTATUS \\ &owner : \mathbb{P} AGENT \\ &spawnLock : \mathbb{P} AGENT \\ &bindLock : \mathbb{P} AGENT \\ &phase : AGENT \rightarrow PHASE \end{aligned} $
	$ \begin{aligned} &\#owner \leq 1 \\ &\#spawnLock \leq 1 \\ &\#bindLock \leq 1 \\ &socketStatus = sabsent \Rightarrow owner = \emptyset \\ &socketStatus = sstale \Rightarrow owner = \emptyset \\ &socketStatus = sbound \Rightarrow \#owner = 1 \\ &socketStatus = slistening \Rightarrow \#owner = 1 \end{aligned} $

owner is the singleton (or empty) set naming the agent whose socket is currently installed at the path; a lock component names its current holder. Modelling these as sets of size at most one lets the empty set stand for “none” without an extra sentinel value.

5 Initialization

<i>Init</i>	
<i>Sys'</i>	$ \begin{aligned} &socketStatus' = sabsent \\ &owner' = \emptyset \\ &spawnLock' = \emptyset \\ &bindLock' = \emptyset \\ &phase' = (\lambda a : AGENT \bullet idle) \end{aligned} $

A single initial state: no file, no locks held, every agent idle.

6 Lock Operations

The two locks are acquired non-blockingly: an acquire is *enabled* only when the lock is free, so a “blocked” agent is simply an agent whose acquire is not enabled. Cyclic waiting — and hence deadlock — is what ProB’s deadlock check must rule out.

The lock order is spawn-outer, bind-inner. An agent that already holds the bind lock never asks for the spawn lock, and vice versa: each agent holds at most one of the two. This is encoded directly in the acquire guards, and is the syntactic reason no cyclic wait can form.

<i>AcquireBindLock</i>	
ΔSys	
$a? : AGENT$	
	$ \begin{aligned} &phase\ a? = idle \\ &a? \notin spawnLock \\ &bindLock = \emptyset \\ &bindLock' = \{a?\} \\ &phase' = phase \oplus \{a? \mapsto hasBind\} \\ &socketStatus' = socketStatus \\ &owner' = owner \\ &spawnLock' = spawnLock \end{aligned} $

ReleaseBindLock ΔSys $a? : \text{AGENT}$
$a? \in \text{bindLock}$ $\text{phase } a? = \text{listening}$ $\text{bindLock}' = \emptyset$ $\text{phase}' = \text{phase} \oplus \{a? \mapsto \text{serving}\}$ $\text{socketStatus}' = \text{socketStatus}$ $\text{owner}' = \text{owner}$ $\text{spawnLock}' = \text{spawnLock}$

Releasing the bind lock is the last step of the critical section: the agent has bound and listened, and now becomes the running display.

AcquireSpawnLock ΔSys $a? : \text{AGENT}$
$\text{phase } a? = \text{idle}$ $a? \notin \text{bindLock}$ $\text{spawnLock} = \emptyset$ $\text{spawnLock}' = \{a?\}$ $\text{phase}' = \text{phase} \oplus \{a? \mapsto \text{hasSpawn}\}$ $\text{socketStatus}' = \text{socketStatus}$ $\text{owner}' = \text{owner}$ $\text{bindLock}' = \text{bindLock}$

ReleaseSpawnLock ΔSys $a? : \text{AGENT}$
$a? \in \text{spawnLock}$ $\text{phase } a? = \text{reaping}$ $\text{spawnLock}' = \emptyset$ $\text{phase}' = \text{phase} \oplus \{a? \mapsto \text{exited}\}$ $\text{socketStatus}' = \text{socketStatus}$ $\text{owner}' = \text{owner}$ $\text{bindLock}' = \text{bindLock}$

7 The Bind Critical Section

Every operation in this section is guarded by $a? \in \text{bindLock}$: in the current design the probe, the stale cleanup, the bind, and the listen all run under the one held bind lock. That single guard is the whole serialisation; removing it from **CleanupStale** and **Bind** yields the buggy variant of Section 11.

Probe records that the agent has observed the path. Observation and the action taken on it are separate steps, so the state may change between them — this is the time-of-check/time-of-use gap the bind lock must close.

<i>Probe</i>
ΔSys
$a? : AGENT$
$a? \in bindLock$ $phase\ a? = hasBind$ $phase' = phase \oplus \{a? \mapsto probed\}$ $socketStatus' = socketStatus$ $owner' = owner$ $spawnLock' = spawnLock$ $bindLock' = bindLock$

CleanupStale unlinks a socket that probes *dead*: a stale leftover (**sstale**) or — and this is the dangerous case — one that is bound but not yet listening (**sbound**). It never touches a live (**slistening**) socket. A probing agent never owns the path (it has not bound), so this only ever removes another agent's file.

<i>CleanupStale</i>
ΔSys
$a? : AGENT$
$a? \in bindLock$ $phase\ a? = probed$ $socketStatus \in \{sstale, sbound\}$ $socketStatus' = sabsent$ $owner' = \emptyset$ $phase' = phase$ $spawnLock' = spawnLock$ $bindLock' = bindLock$

Under the bind lock this operation's **sbound** branch is unreachable: the only agent that could hold the lock is **a?** itself, and it has not bound, so no socket is in the bind window. ProB confirms the branch never fires (Section 10). In the buggy variant it fires, and that is precisely the two-winner defect.

<i>Bind</i>
ΔSys
$a? : AGENT$
$a? \in bindLock$ $phase\ a? = probed$ $socketStatus = sabsent$ $socketStatus' = sbound$ $owner' = \{a?\}$ $phase' = phase \oplus \{a? \mapsto bound\}$ $spawnLock' = spawnLock$ $bindLock' = bindLock$

Bind is atomic and succeeds only from **sabsent**. A concurrent binder that finds the path already taken loses the race:

BindFail

ΔSys

$a? : AGENT$

$a? \in bindLock$
 $phase\ a? = probed$
 $socketStatus = sbound$
 $a? \notin owner$
 $phase' = phase \oplus \{a? \mapsto lostRace\}$
 $bindLock' = \emptyset$
 $socketStatus' = socketStatus$
 $owner' = owner$
 $spawnLock' = spawnLock$

Listen promotes the bound socket to live. An agent listens on its *own* descriptor: if it still owns the path the path goes live; if its file was unlinked from under it (the buggy variant), it listens on an orphaned descriptor and the path's status is unchanged.

Listen

ΔSys

$a? : AGENT$

$a? \in bindLock$
 $phase\ a? = bound$
 $phase' = phase \oplus \{a? \mapsto listening\}$
 $socketStatus' =$
 if $a? \in owner$ **then** $slistening$ **else** $socketStatus$
 $owner' = owner$
 $spawnLock' = spawnLock$
 $bindLock' = bindLock$

An agent that probed a *live* path concedes without binding:

LoseRaceLive

ΔSys

$a? : AGENT$

$a? \in bindLock$
 $phase\ a? = probed$
 $socketStatus = slistening$
 $phase' = phase \oplus \{a? \mapsto lostRace\}$
 $bindLock' = \emptyset$
 $socketStatus' = socketStatus$
 $owner' = owner$
 $spawnLock' = spawnLock$

A lost racer exits, changing nothing but its own phase:

Exit

ΔSys

$a? : AGENT$

$phase\ a? = lostRace$
 $phase' = phase \oplus \{a? \mapsto exited\}$
 $socketStatus' = socketStatus$
 $owner' = owner$
 $spawnLock' = spawnLock$
 $bindLock' = bindLock$

8 Failure and Teardown

An owner may crash, leaving a stale file. Killing the process releases any lock it held:

<i>Crash</i>
ΔSys
$a? : AGENT$
$a? \in owner$ $phase\ a? \in \{listening, serving\}$ $socketStatus' = sstale$ $owner' = \emptyset$ $phase' = phase \oplus \{a? \mapsto exited\}$ $bindLock' = bindLock \setminus \{a?\}$ $spawnLock' = spawnLock$

A reaper terminates the live owner under the spawn lock, then clears the files. The file-clearing step needs the bind lock free (it mirrors `reap's _clear_dead_files_locked`), so a concurrent binder is never unlinked mid-bind:

<i>Reap</i>
ΔSys
$a? : AGENT$
$a? \in spawnLock$ $phase\ a? = hasSpawn$ $socketStatus = slistening$ $bindLock = \emptyset$ $phase' =$ $(phase \oplus (owner \times \{exited\}))$ $\oplus \{a? \mapsto reaping\}$ $socketStatus' = sabsent$ $owner' = \emptyset$ $spawnLock' = spawnLock$ $bindLock' = bindLock$

When there is no live owner, the reaper simply clears any dead file, again only with the bind lock free:

<i>ReapDead</i>
ΔSys
$a? : AGENT$
$a? \in spawnLock$ $phase\ a? = hasSpawn$ $socketStatus \neq slistening$ $bindLock = \emptyset$ $socketStatus' = sabsent$ $owner' = \emptyset$ $phase' = phase \oplus \{a? \mapsto reaping\}$ $spawnLock' = spawnLock$ $bindLock' = bindLock$

An exited agent may re-enter as a fresh process. This keeps the system live — it never runs out of enabled operations at a benign terminal state — so that the deadlock check reports only genuine lock deadlocks, never mere termination:

Reset

ΔSys

$a? : AGENT$

$phase\ a? = exited$

$a? \notin spawnLock$

$a? \notin bindLock$

$phase' = phase \oplus \{a? \mapsto idle\}$

$socketStatus' = socketStatus$

$owner' = owner$

$spawnLock' = spawnLock$

$bindLock' = bindLock$

9 Invariants

Five properties must hold across all reachable states. They are stated here as predicates over **Sys**; how each is discharged is given in Section 10. None is placed in the **Sys** schema: a predicate placed there would be enforced as a guard on every successor, silently disabling any operation that would break it and thereby masking the very defect this model exists to catch.

Invariant 1 — singleton serving. At most one agent is in **serving**:

$$\#\{a : AGENT \mid phase\ a = serving\} \leq 1.$$

Invariant 2 — never unlink live. No operation carries **socketStatus** from **slistening** to **sabsent** except a legitimate **Reap** of that owner. This is a property of the transition relation, established by construction: the only operations that assign **socketStatus'** = **sabsent** are **CleanupStale**, **Bind**'s pre-condition path, **Reap**, and **ReapDead**. Their guards are, respectively, **socketStatus** $\in \{sstale, sbound\}$; a start from **sabsent**; **socketStatus** = **slistening**; and **socketStatus** $\neq$ **slistening**. Of these, only **Reap** starts from **slistening**, and it does so only after terminating the owner. **CleanupStale**'s guard excludes **slistening** outright, so a live socket is never unlinked.

Invariant 3 — no two winners. At most one agent holds a non-absent claim on the path:

$$\#\{a : AGENT \mid phase\ a \in \{bound, listening, serving\}\} \leq 1.$$

Invariant 4 — deadlock freedom. No reachable state leaves every agent unable to progress while some agent is non-terminal. The two-lock order (spawn-outer, bind-inner, never both held by one agent, an agent holding one never requesting the other) makes a cyclic wait impossible.

Invariant 5 — lost-racer clean exit. An agent in **lostRace** can only reach **exited**, via **Exit**, which changes nothing but its own phase. By construction **lostRace** is produced only by **BindFail** and **LoseRaceLive** and consumed only by **Exit**; the **Exit** schema frames **socketStatus**, **owner**, **spawnLock**, and **bindLock** unchanged.

10 Verification

Type-checking is a fuzz pass:

```
fuzz -t docs/display_lifecycle.tex
```

Invariants 1 and 3 are state properties, checked by asking ProB whether their negation is *reachable*. A goal that is not found means the invariant holds on every reachable state:

```
# Invariant 1 (must report: goal NOT found)
probccli docs/display_lifecycle.tex -model_check -nodead \
  -goal "card(\{a|a:AGENT & phase(a)=serving\})>1" \
  -p DEFAULT_SETSIZE 2
```

```
# Invariant 3 (must report: goal NOT found)
probcli docs/display_lifecycle.tex -model_check -nodead \
  -goal "card({a|a:AGENT & phase(a):{bound,listening,serving}})>1" \
  -p DEFAULT_SETSIZE 2
```

Invariant 4 is the deadlock check over the full reachable state space:

```
# Invariant 4 (must report: no deadlock)
probcli docs/display_lifecycle.tex -model_check \
  -p DEFAULT_SETSIZE 2
```

Invariants 2 and 5 are transition/structural properties, established by the shape of the operation schemas as argued in Section 9 and corroborated by ProB’s operation-coverage report (`CleanupStale` never fires from a `slistening` state; `Exit` is the sole consumer of `lostRace`).

11 Fidelity: reproducing the two-winner defect

A model that cannot reproduce the known defect proves nothing. The buggy variant is the current specification with one change: the bind-lock serialisation is removed from the critical section, so cleanup and bind no longer run under a single held lock. Concretely, delete the guard `a? ∈ bindLock` from `CleanupStale` and `Bind` (and, to let a second agent enter the window, from `Probe` and `Listen`). The two-winner interleaving then becomes reachable:

1. Agent x probes the absent path and binds it: `socketStatus = sbound`, `owner = {x}`, `phase x = bound`.
2. Agent y probes during the bind window and reads `sbound` as dead.
3. Agent y runs `CleanupStale`, unlinking x ’s fresh socket: `socketStatus = sabsent`, `owner = ∅` — while `phase x` is still `bound`.
4. Agent y binds its own socket: `owner = {y}`, `phase y = bound`.
5. Now `phase x = bound` and `phase y = bound`: two agents hold a non-absent claim, violating Invariant 3.

Under the intact bind lock this interleaving is impossible: step 2 cannot occur because y cannot acquire the bind lock while x holds it, and x holds it from before step 1 until after it has listened and released. The reachability search for Invariant 3 therefore returns *goal not found* for the current design and *goal found* — the trace above — for the buggy variant. That difference is the evidence that the model detects the race the code was changed to prevent.